Contents

app_2 – the caOS bootloader: user guide **1**
1. Flash map . . . 1
2. Images . . . 1
   2.1 The `.cimg` format . . . 1
   2.2 Building an image . . . 2
   2.3 Keys . . . 2
3. Boot . . . 2
   3.1 The channel between bootloader and image . . . 3
4. Updating . . . 3
   4.1 From recovery . . . 3
   4.2 From recovery, over USB (DFU) . . . 3
   4.3 From the running image . . . 3
5. The image side (`EN_BOOT_IMAGE`) . . . 4
6. Bootloader CLI . . . 4
7. Full test . . . 4
8. Known limits . . . 5

app_2 – the caOS bootloader: user guide

`app_2` is the caOS bootloader for the **NUCLEO-H563ZI** and **NUCLEO-H753ZI**. It is an ordinary caOS application: it uses the drivers, network, flash, SHA-256 and Ed25519 of the libraries, so anyone who has the package can build their own.

What it does:

- at reset it picks a slot, checks its image (SHA-256 and signature) and hands the CPU over with `ca_os_end()`, which stops caOS and puts the hardware back as after a reset;
- it receives new images over the network (TFTP: the PC pushes them), from the running image too, or over USB (DFU, `dfu-util`) while in recovery;
- it starts a new image **on trial**, with the IWDG armed; if the image does not confirm within three starts, it goes back to the previous one (**rollback**).

---

1. Flash map

The same on both boards (2 MB in two 1 MB banks):

Area	Address	Size	Notes
bootloader (app_2)	`0x08000000`	256 KB	bank 1, does not update itself; write protection with `boot protect on`
slot A	`0x08040000`	768 KB	bank 1
slot B	`0x08100000`	768 KB	bank 2
boot configuration	last sector of bank 2	8 KB (H563) / 128 KB (H753)	written by the bootloader only

The map is shared with the images in `caos/inc/boot_image.h` (`CA_BOOT_SLOT_A_BASE`, ...): a single source for both sides.

2. Images

2.1 The `.cimg` format

A slot holds a 1 KB header followed by the image, linked to start right after it (the vector table stays aligned as VTOR requires):

Byte	Field	Content
0	magic	`'CAIM'`
4	u16	header version (1)
6	u16	flags: b0 = signed
8	u32	image size
12	u32	address of the vector table = slot base + 0x400
16	u32	version
20	char[32]	tag, free text
52	u32	build time (epoch)
56	u8[32]	SHA-256 of bytes 0..55 + the image
88	u8[64]	Ed25519 signature of bytes 0..87 (the SHA in it covers the image)

2.2 Building an image

A workspace application becomes an image by adding `image.cmake`:

```
set(CAOS_IMAGE_SLOTS A B)
```

The build produces `<mcu>_<app>_slotA.elf` and `_slotB.elf`, each linked into its own slot. Then:

```
./build_caos.sh H563 -wp
python3 tools/caos_image.py elfs/H563/h563_app_4_slotB.elf --version 3 --tag "v1.3" --sign
python3 tools/caos_image.py --info elfs/H563/h563_app_4_slotB.cimg
```

An image linked for A does not start from B: the bootloader checks the address in the header.

2.3 Keys

The repository holds a **demo** key pair: `workspace/app_2/boot_ed25519.pem` (private, public on purpose) and `boot_pubkey.h` (the public half, compiled into the bootloader). With it anyone can sign images for the example. A real product makes its own pair:

```
python3 tools/caos_image.py --keygen --key ~/secret/prod.pem   # regenerates boot_pubkey.h
python3 tools/caos_image.py <slot.elf> --sign ~/secret/prod.pem
```

With `BOOT_REQUIRE_SIGNED 1` (in `boot_slot.h`, the default) only images signed with that key start.

3. Boot

At reset, before any task exists, the bootloader:

1. **confirm** – if the slot on trial marked itself confirmed, it becomes the active slot;
2. **update request** – if the image asked to try the other slot, it checks it and makes it `pending`;
3. **trial** – if a slot is `pending`: it counts one try; beyond `CA_BOOT_MAX_TRIES` (3) it **rolls back**: erases the slot and goes back to the active one. Otherwise it arms the IWDG (`CA_BOOT_IWDG_MS`, 8 s) and starts the slot on trial;
4. **normal boot** – the active slot if valid, else the first valid one;
5. **recovery** – no valid image, or B1 held at reset: the bootloader stays up with the console, the CLI and the network.

Erasing the slot at rollback is not a detail: without it, the "first valid" rule could start it again without the IWDG on the day the other slot fails.

3.1 The channel between bootloader and image

Bootloader and image share no memory: the only channel is two **backup registers** (`bkp.h`), which survive the reset between them.

Register	Written by	Content
`CA_BOOT_BKP_STATE` (16)	bootloader; the image when confirming	magic, slot on trial, tries (`0xFF` = confirmed)
`CA_BOOT_BKP_MSG` (17)	image	update request for a slot

The confirmation is in the state register and not in the message one: a confirmed image that then asks for an update must not lose its confirmation.

! On the Nucleo boards **VBAT is tied to VDD**: removing the power clears the backup registers. An image on trial then starts again from zero tries, and one confirmed but not yet recorded is tried again (and confirms again). Neither case is dangerous; with VBAT supplied by a battery the problem does not exist.

4. Updating

4.1 From recovery

The bootloader in recovery gets an address by DHCP and prints it:

```
BOOT: update - tools/caos_tftp put 192.168.10.93 <image.cimg>
```

From the PC (standard library, no sudo: port 69 is the board's):

```
python3 tools/caos_tftp put 192.168.10.93 elfs/H563/h563_app_4_slotA.cimg
```

The bootloader accepts only `.cimg` names, writes into the slot that would **not** start at reset, checks the image, makes it `pending` and resets into the trial:

```
BOOT: update "h563_app_4_slotA.cimg" from 192.168.10.1 -> slot A, erasing ...
BOOT: update OK, slot A v1 "app_4 A", signed - pending, reset into its trial
BOOT: trial of slot A, start 1 of 3, IWDG 8000 ms
```

A tampered image, or one linked for the other slot, is received but declared not valid, and does not start.

4.2 From recovery, over USB (DFU)

Recovery also exposes a **DFU interface in download mode** (`ca01:0040`, "caOS bootloader") on the USB port used by example_57 (H563: USB_DRD; H753: OTG_FS). With `dfu-util` no network is needed:

```
dfu-util -l                                    # Found DFU: [ca01:0040] ... name="caOS
    bootloader"
dfu-util -d ca01:0040 -a 0 -D elfs/H753/h753_app_4_slotA.cimg
```

The chosen slot, the check and the trial are the same as with TFTP. An image that is not valid ends with `dfuERROR, status(7) = Programmed memory failed verification` on the PC. 64-byte blocks, one per EP0 packet: about 12 s for 117 KB on the H563, 45 s for 120 KB on the H753 (128 KB sectors), erase included. One update at a time: TFTP and DFU exclude each other.

4.3 From the running image

An image with `EN_BOOT_IMAGE` receives the new image into the **other** slot and asks the bootloader to try it (`ca_boot_receive_update()`); see par.5. The command on the PC is the same. An image still on trial and not confirmed refuses the update: the other slot is its way back.

5. The image side (EN_BOOT_IMAGE)

caos/inc/boot_image.h, flag EN_BOOT_IMAGE (requires EN_WDT):

```
char ca_boot_image_slot(void);       /* 'A', 'B', 0 = not started from a slot */
bool ca_boot_is_trial(void);         /* started as a trial, not confirmed yet */
void ca_boot_image_start(void);      /* in ca_os_main: on a trial, take the IWDG over */
void ca_boot_confirm(void);          /* healthy: keep this slot */
bool ca_boot_request_update(char slot);              /* resets */
bool ca_boot_receive_update(ca_mac_t mac, u32 ms);   /* TFTP into the other slot, resets */
```

ca_boot_image_start() reconfigures the IWDG left running by the bootloader and feeds it from a job: for CA_BOOT_TRIAL_MS (60 s) if the image does not confirm, for ever after ca_boot_confirm(). An image that does not call ca_boot_image_start() does not feed the IWDG and is reset after 8 s: that is the rollback case.

When to confirm is the application's decision, and the most important one: it must mean "I really work". workspace/app_4, the demo image, confirms when the network is up (DHCP lease obtained).

```
void ca_os_main(void)
{
    ca_boot_image_start();
    /* ... jobs ... */
    ca_os_run(true);
}

/* in a task, once the application is healthy */
ca_boot_confirm();
```

6. Bootloader CLI

```
boot info          slots, conf (active, pending), reason for recovery
boot select A|B    active slot at the next reset (no trial)
boot go [A|B]      starts a slot now
boot erase A|B     erases a slot
boot protect [on|off]  write protection of the bootloader area (option bytes)
tftp recv [sec]    receives a file and discards it (network check)
```

boot protect on writes the option bytes (H563: WRP groups 0-7 of FLASH_WRP1R; H753: sectors 0-1 of FLASH_WPSN1): the protection stays across resets and power cycles, and the banner shows it. **It blocks SWD programming too** until boot protect off. The symptom depends on the family: on the H753 prog.sh fails at erase (failed to erase memory; reset_mcu.sh may be needed afterwards), on the H563 the erase silently skips the protected sectors and the error comes at verify (Data mismatch). On the H563, therefore, rewriting *the same* bootloader "succeeds": the bytes are already those. Alternatively, clear the protection option bytes from STM32CubeProgrammer.

app_4 adds img info (slot, trial, state register) and img confirm.

7. Full test

workspace/app_2/test_boot_pc.py runs the whole cycle on one board and ends with a verdict:

```
./build_caos.sh H563 -wp
sudo -n ./tools/bench_net.sh up            # bench network: at HOME only
python3 workspace/app_2/test_boot_pc.py --mcu H563 --tty /dev/serial/by-id/...-if02
```

Phase	What it checks
1	recovery with empty slots and a DHCP address
2	tampered image not valid (SHA-256), non .cimg name refused

Phase	What it checks
3	app_4 in A: trial with IWDG, confirmation
4	update from the running image: A recorded as active, B on trial and confirmed
5	image that never confirms (app_1): three trials, rollback, A erased, B starts again

Result on 21/10/2026: **5/5 on NUCLEO-H563ZI and on NUCLEO-H753ZI**. Tested by hand on both: DFU (good image -> trial and confirmation; tampered -> `errVERIFY`) and `boot protect` (on -> SWD blocked, bootloader intact; off -> SWD possible again). DFU speed: ~10 KB/s on the H563, ~2.7 KB/s on the H753.

8. Known limits

- The bootloader does not update itself: a defect in the bootloader is fixed over SWD (with `boot protect off` if the protection is on).
- No downgrade protection: an older version, if signed, is accepted (on purpose).
- Anyone on the network can push a file: the signature is what prevents starting it.